OFFBYONE 2026

Splitting the AI Brain

Discovering V8 Sandbox Bypasses with Multi-Agent Collaboration

Speakers: Jiesong Wu & Haohao Chen

ABOUT US



Jiesong Wu (Picasso)
Security Researcher
JD.COM
@PicassoLay

Haohao Chen
Security Researcher
JD.COM
@TheDog0402

Hongze Shang
Security Researcher
JD.COM
@hzshang15

Yinyi Wu
Security Researcher
JD.COM
@_3ndy1

The V8 Sandbox matured

What our fuzzing setup told us, in three moments

2024

The sandbox is new. Early fuzzing campaigns produce crashes in bulk.

late 2025

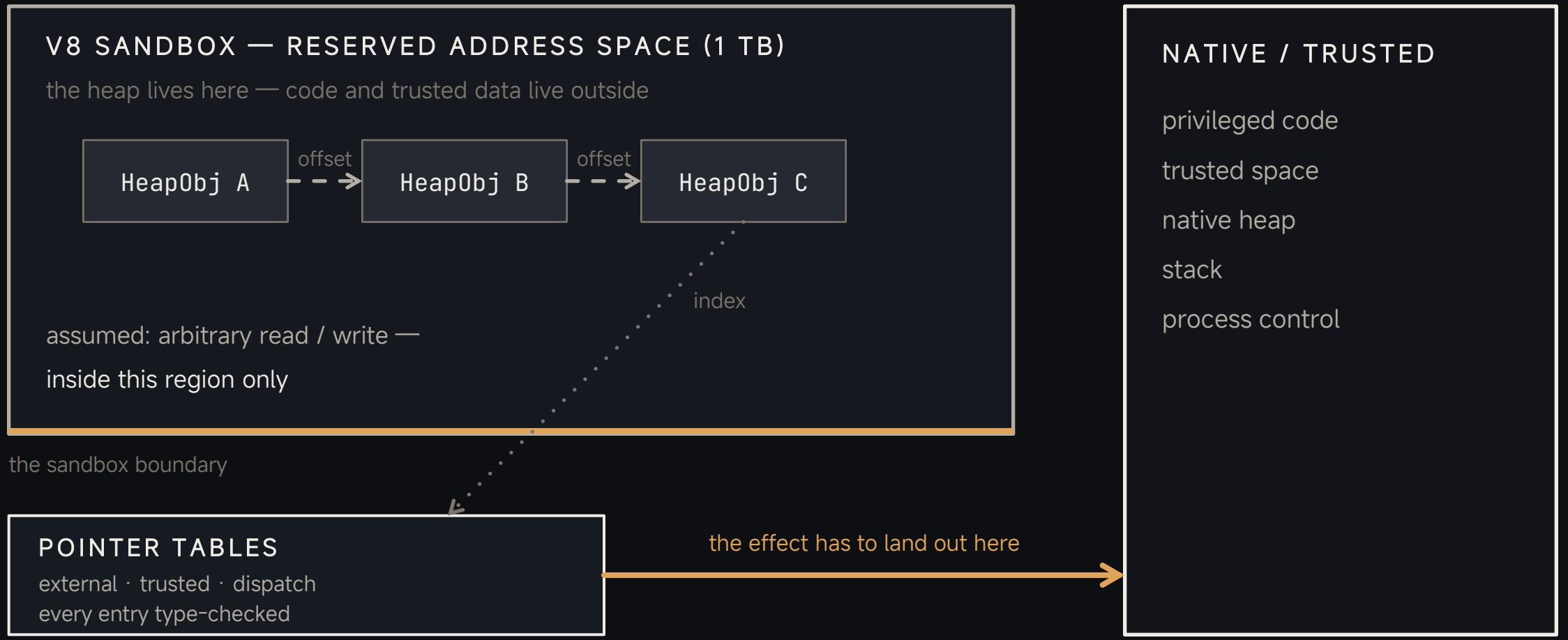
We replay the old corpus on the then-current build. One UAF still reproduces.

later

No new sandbox violation found by fuzzer.

What counts as a sandbox bypass

In-sandbox corruption alone does not establish a bypass.



Adapted from Samuel Groß, “JS Engine Security in 2025,” Power of Community 2025

Why random corruption lost its leverage

Whether random corruption becomes useful depends on four factors lining up.

ASSUMED STARTING POINT

```
arbitrary read / write inside the sandbox
```

1

the right object

An object whose state is read by trusted or native code.

2

the right field

A field the trusted side will read back and act on.

3

a value with meaning

Not garbage — a state the consuming code interprets.

4

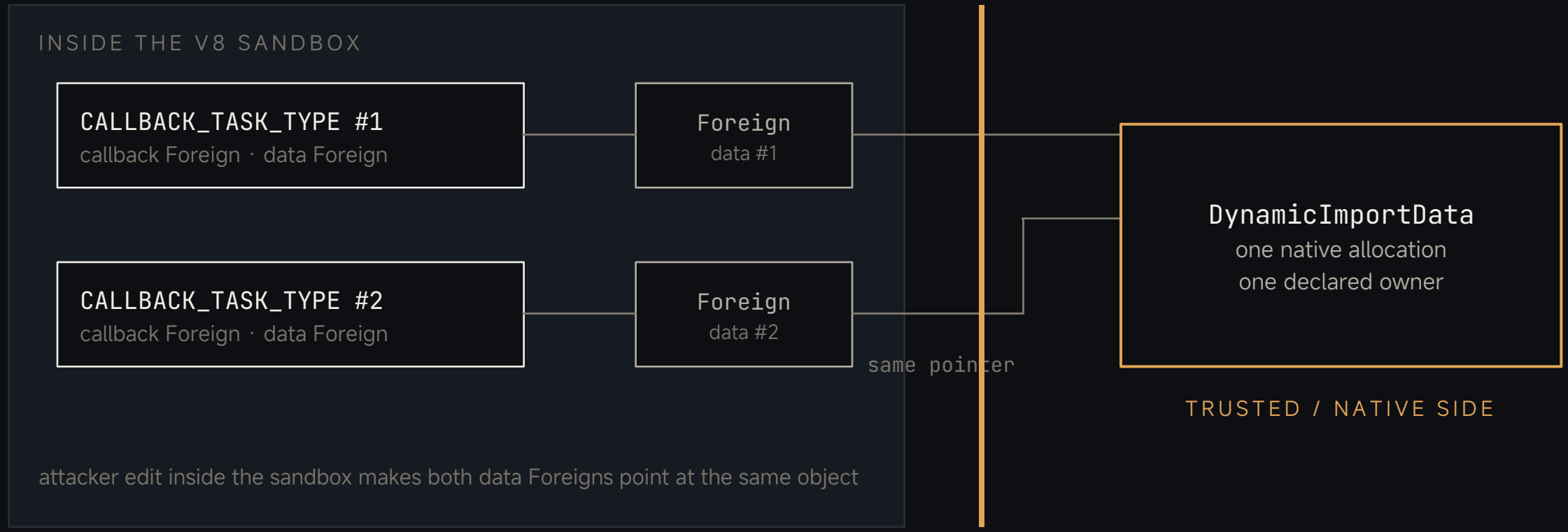
consumed in time

The corrupted state is consumed at the right execution point.

Random mutations rarely align the right object, field, value, and execution timing.

The crash that survived

d8 dynamic import — two callback tasks, two data Foreigns, one shared native object.



Two handles into one allocation.

LIFETIME

The crash that survived

d8 dynamic import — the free is callback-driven

01 · SETUP

Two tasks, one object

each callback task carries its own data Foreign

both Foreigns hold the same DynamicImportData*

→

02 · FREE

First callback frees it

task #1 finishes and calls DeleteDynamicImportData()

the 56-byte object is deleted — task #2 still queued

→

03 · USE

Second callback uses it

task #2 enters DoHostImportModuleDynamically()

the same pointer is passed in and read — use-after-free

ASAN, CONDENSED

```
AddressSanitizer: heap-use-after-free — READ of size 8
used    Shell::DoHostImportModuleDynamically      d8.cc:1672
freed   DeleteDynamicImportData (operator delete)   d8.cc:1695
```

One allocation, one free — and a second user that was never told.

The same invariant, another Foreign type

The hypothesis came from the researcher; the search was done by the model.

THE HUMAN QUESTION

“Do other Foreign types share this lifetime error?”

The seed bug abstracts into an invariant:
several sandbox-visible Foreigns, one native object — freed on one path, still used on another.
Choosing that question is researcher work.

THE MACHINE SEARCH

Searched Foreign types with native backing allocations, checked each against the invariant.

SEED · D8 DYNAMIC IMPORT

`Foreign ×2 → DynamicImportData`

freed by the first callback, used by the second

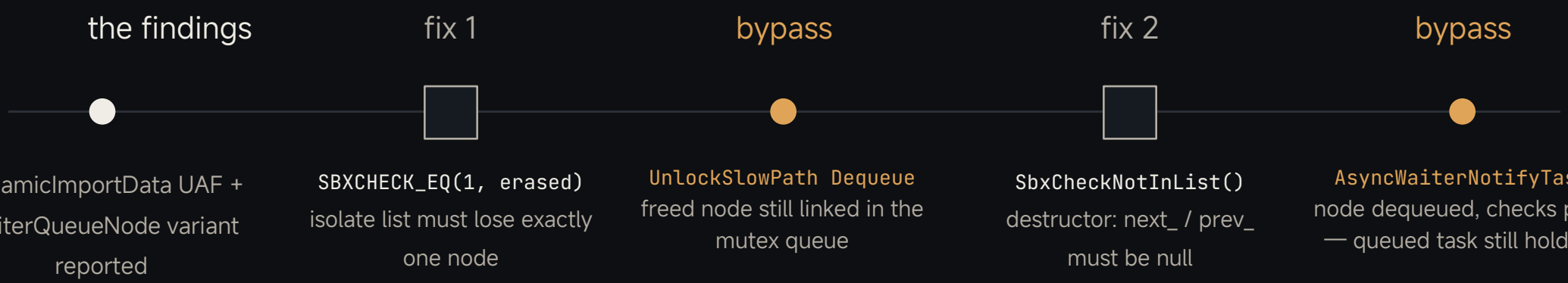
VARIANT · ATOMICS.MUTEX LOCKASYNC

`Foreign ×2 → WaiterQueueNode`

freed during one unlock, accessed again during another

One invariant, a second instance — a native object freed on one path, still used on another. This division of labor becomes the system later in this talk.

Patch history and subsequent bypasses



Patch 1 verified removals from the isolate list; the freed node was still linked in the mutex queue. Patch 2 verified the link pointers; the node was already dequeued — with a notify task pending.

THE DIVISION OF LABOR

Human-guided variant hunting

How the work was split between the researcher and the model.

HUMAN

Extract the invariant
from one crash to a general lifetime error

Choose the question
which directions are worth investigating

Judge the result
is this chain real, or a plausible story

THE MODEL

Search related implementations
other Foreign-backed types and their lifetime paths

Trace the chains
callers, owners, GC paths, destructors

Instantiate the pattern
one invariant, many candidate objects

We ran this loop by hand before automating it.

SCOPE

From one object to function-level tasks

The unit of work became a single function.

BEFORE — MANUAL VARIANT HUNTING

one object type at a time
Foreign, then maybe the next Foreign

one invariant at a time
the lifetime error we already knew

candidates picked by hand
breadth limited by researcher hours

→

AFTER — SCANNING AS A PIPELINE

`scoped files under src/`
functions extracted from them — each unit is a task

a minimal task package
target file and function, audit instructions, repository access

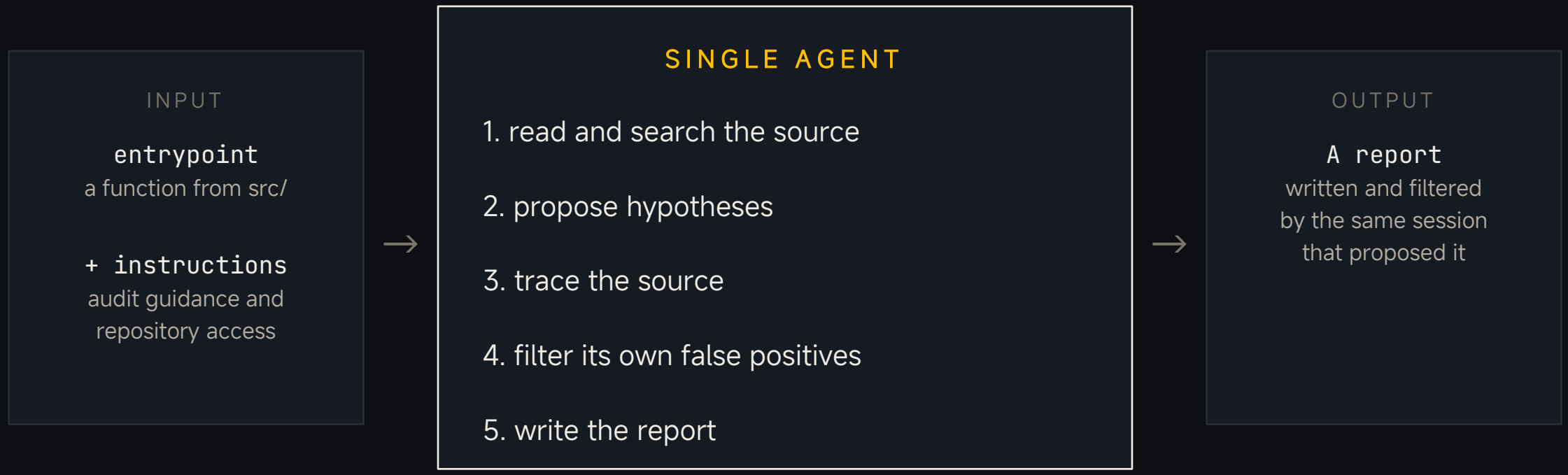
the agent searches the repo itself
functions are entry points, not the whole scope

One function, one task — the agent assembles its own context.

FIRST ATTEMPT

The monolithic agent

One session carried the whole audit, start to finish.



Proposing, tracing, and judging all lived in one context window.

EVIDENCE

An Intl UAF from batch auditing

Detected in a batch audit from the single-agent scanner era.

SCAN REPORT — KEY EXCERPTS

```
function
NumberFormatPrototypeFormatNumber
cached raw pointer
fmt = icu_number_formatter()->raw()
re-entry window
ToPrimitive() → attacker valueOf()
inside the callback
field replaced, GC triggered
back in C++
stale fmt reused in FormatNumeric()
```

CALL SEQUENCE

1. JS calls the bound format(value)
2. C++ caches the raw ICU formatter pointer
3. ToPrimitive(value) re-enters JavaScript
4. valueOf() replaces the field, triggers GC
5. the formatter is freed inside the window
6. C++ resumes — stale pointer passed to ICU

A single-agent audit reached a real, new vulnerability.

AN INFORMAL CHECK

Could it rediscover a harder bug?

The Intl find proved it could find real bugs. How far does that ability go?

> WHERE IT STOOD
>
> The Intl find was real proof: the agent could find sandbox bugs. What it didn't show was where that ability stops.

> THE CHECK
>
> Pick a UAF disclosed at the time — subtle, already fixed. Point the single agent at the vulnerable revision, with no hint about which bug we mean.

The first run missed it. One broad hint changed the result.

THE OBSERVED DIFFERENCE

One extra sentence changed the audit

Same model, same code unit, same task. One prompt had a second sentence.

GENERIC AUDIT PROMPT

`“Audit this function for sandbox-boundary violations.”`

RESULT

Missed the real bug.

SAME PROMPT + ONE SENTENCE

`“Audit this function for sandbox-boundary violations. Focus on UAF.”`

RESULT

Traced the source to the bug.

Recovered the disclosed root cause and trigger chain.

With the added UAF hint, the model identified the bug in this attempt.

Prompts shown are simplified illustrations — the original conversation was not preserved; both runs used the same model and code unit.

What this suggested for the design

Could a dedicated agent propose directions for later audits?

> In the manual loop, choosing the question was the human's job. The model's job started after a direction existed.

FAILURE MODES

One agent carried too much state

Four failures kept repeating in long audit sessions.

Failure 1

It wasted a lot of time expanding the target context.

Failure 2

It finished only part of the hypotheses.

Failure 3

It closed early on the first plausible story.

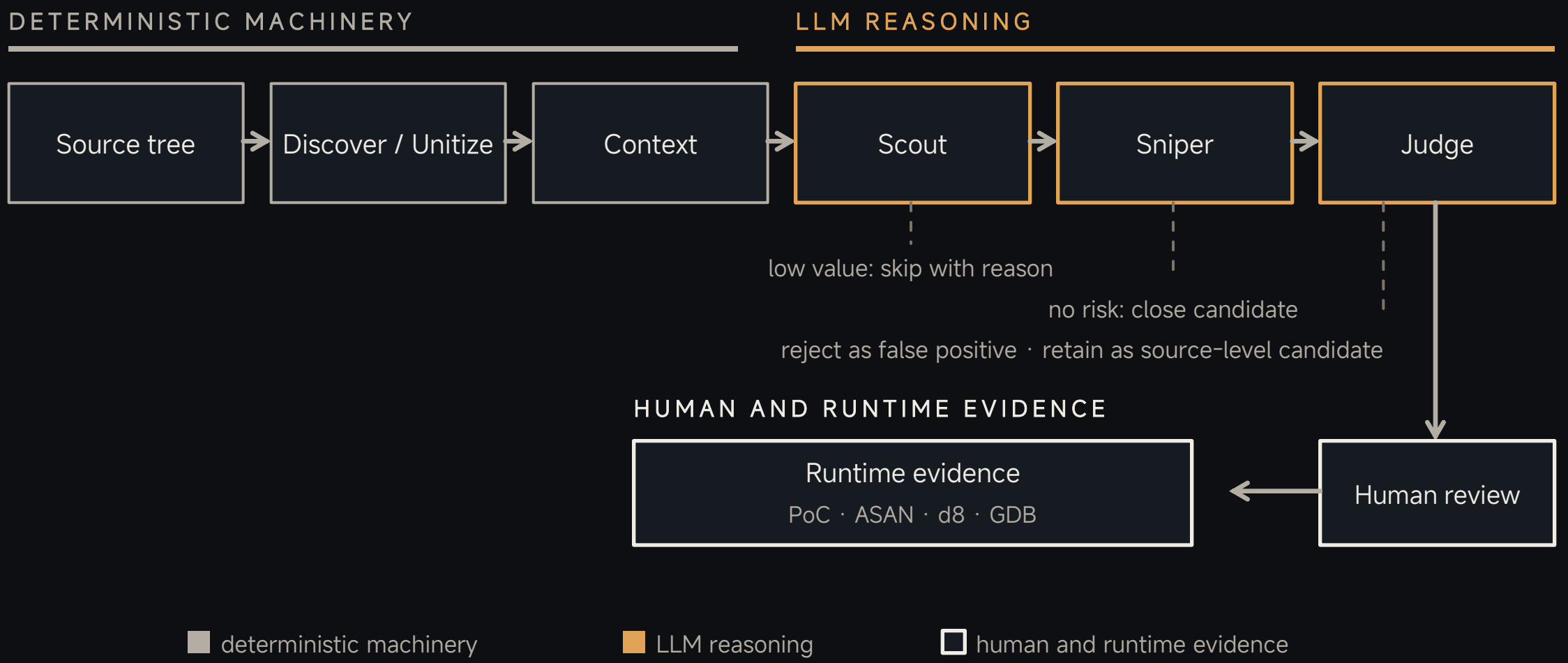
Failure 4

It ignored the false positive rules and still reported the wrong risk.

We split the work to reduce task load and preserve intermediate findings.

THE SYSTEM

Splitting the AI Brain



Cheap stages expand coverage. Expensive stages are reserved for bounded, evidence-bearing questions.

THE SYSTEM

Three agent roles

Each role hands one concrete artifact to the next stage.

Scout

WHAT IT DOES

generates concrete, testable hypotheses — object, field, value, consuming operation, consequence

The template guides the model toward specific hypotheses; it does not guarantee each one is valid.

Sniper

WHAT IT DOES

investigates each hypothesis against the source and reaches a conclusion

The outcome can be a source-backed risk report — or a no-risk conclusion.

Judge

WHAT IT DOES

re-reviews the risk report against the source and applies rule-based filters, looking for counter-evidence

It does not confirm vulnerabilities. Survival means this review found no decisive counter-evidence.

Proposing, building, and attacking no longer share one context window.

Scout turns patterns into testable claims

We use the template to give the next agent a concrete starting point for investigation.

THE PATTERN LIBRARY

22 patterns, written by hand — distilled from dozens of historical issues and prior research experience.

Scout selects, combines, instantiates. A pattern is abstract until it names a concrete object and field.

It may go beyond the library — it can propose hypotheses the library does not contain.

THE OUTPUT — ONE HYPOTHESIS, FIVE FIELDS

OBJECT

`writable MemoryChunk`

FIELD

`untrusted_main_thread_flags_`

VALUE

`set READ_ONLY_HEAP`

CONSUMING OPERATION

`SandboxSafeInReadOnlySpace()`

OUT-OF-SANDBOX CONSEQUENCE

```
a writable page passes as read-only
→ trusted-object processing
```

condensed from Scout output · unit a994ed791c3eb104

How the analysis developed

The same SandboxSafeInReadOnlySpace report, stage by stage.

Scout

3 directions

```
H1  set READ_ONLY_HEAP on a writable page → misclassification
H2  metadata_index_ as an independent native-metadata risk
H3  ReadOnlyPageMetadata reinterpretation without a type check
```

Sniper

source review

```
confirmed — the page's real metadata still passes the isolate
and chunk-ownership checks
traced — HeapVisitor trusted branch → protected-pointer reads
excluded — H2: the mask bounds the table; wrong slots fail the checks
```

Judge

fresh-session review

```
kept the core risk as a source-level candidate
corrected the trigger order: the object must enter the marking
worklist first — otherwise ShouldMarkObject() skips it
```

Review did not just filter the report — it corrected how the bug has to be triggered.

THE FUNNEL

What survives is still a candidate

Totals across archived Judge runs.

340 rejected as false positives, with a recorded reason

112 retained as source-level candidates

each retained candidate required human review and runtime evidence

Surviving Judge review is not a confirmed vulnerability.

The PoC is not in the pipeline. Confirmation is human work, with runtime evidence.

Two findings in detail

GC UAF through forged read-only page identity · BigInt CachedMod OOB write

A writable page accepted as read-only

SandboxSafeInReadOnlySpace() — the check between a sandbox object and trusted-object handling.

WHY THE CHECK PASSES

The metadata is genuinely this chunk's own — with the original index, the isolate and chunk-ownership checks pass.

`static_cast` performs no runtime type check; `ChunkAddress()` reads only base-class fields.

What is never checked: that the page really is read-only.

SOURCE, CONDENSED — memory-chunk.cc / .h

```
// untrusted input — a writable page flag
bool InReadOnlySpace() const {
  return IsFlagSet(READ_ONLY_HEAP); }
// the check
if (!InReadOnlySpace()) return false;
SBXCHECK_EQ(
  static_cast<const ReadOnlyPageMetadata*>(
      Metadata())->ChunkAddress(), address());
return true;  // writable page accepted
```

The checks establish which chunk the metadata belongs to — they do not establish that the page is read-only.

Scout's three hypotheses

Three directions into the same target — and one correction from independent review.

H1	Set `READ_ONLY_HEAP` — the page's own real metadata still passes the `ChunkAddress()` ownership check a writable page is classified read-only	confirmed by Sniper — the core flaw
H2	Corrupt `metadata_index_` into an independent native-metadata risk masked table lookup picks a wrong or stale metadata entry	excluded — the mask bounds the table; wrong slots fail the isolate / chunk checks
H3	The metadata is read through a `static_cast` to `ReadOnlyPageMetadata`, never type-checked is read-only semantics ever verified? — no	confirmed — same root cause as H1

JUDGE'S CORRECTION

Worklist insertion must precede the flag change — otherwise `ShouldMarkObject()` skips the object.

A live table entry pointing to a freed object

The researcher's follow-up construction on the same flaw — two liveness states drift apart.



The handle was kept alive. The object behind it was not.

Follow-up human construction — flaky by nature: a worker bit-flip plus a precise heap layout.

What the run record shows

Static evidence from our follow-up verification

RUN LOG, CONDENSED

```
V8 sandbox violation detected
— a read access; needs manual investigation
Received signal 11 SEGV_MAPERR
414141414141


via WasmDispatchTable::offheap_data
    DispatchTableClear · WasmTableObject::Set
```

The object confusion leads to a read from 0x414141414141. This log alone does not demonstrate arbitrary write or PC control.

WHAT THE CRASH IS

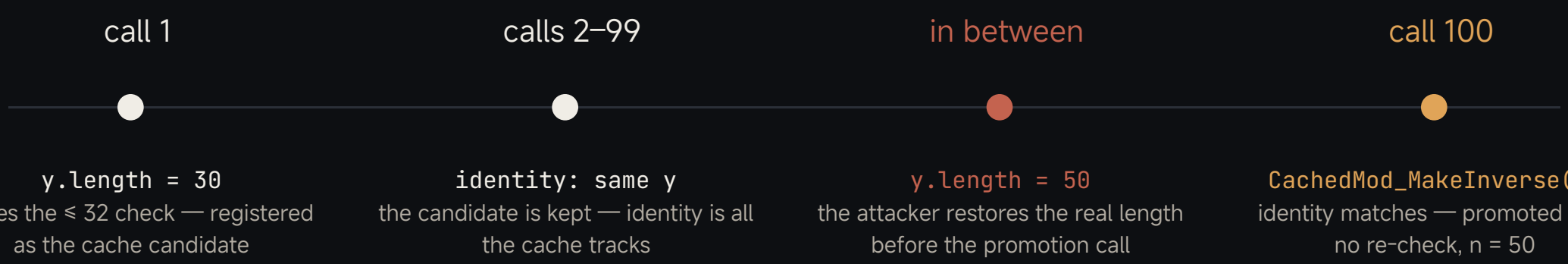
The confused pair: a table entry that still names a WasmDispatchTable, memory that now holds an IrRegExpData. Clearing entry 0 reads offheap_data through the wrong object's fields.

> The agents identified the read-only classification flaw; our follow-up work used it to construct a trusted-space UAF.

Flaky — it needs many tries. Never formally submitted; upstream later hardened this whole area.

BigInt divisor caching across calls

MutableBigInt_AbsoluteModAndCanonicalize — the CachedMod fast path.



The cache remembers object identity — not a trusted length snapshot.

The cached path assumes n never exceeds 32

`kMaxCachedModDivisorSize = 32 · kSmallScratchSize = 100 digits`

THE CONSTANTS, THEN ONE WORKED EXAMPLE

```
kMaxCachedModDivisorSize = 32
kSmallScratchSize = 100
static_assert(100 ≥ 32*3 + 1)
```

worked example — n = 50

```
A.len() = 100 · inv.len() = 51
scratch_space = A.len() + inv.len() = 151
// = 3n+1 here, because A.len() = 2n
physical capacity = 100 digits
```

CONDENSED SOURCE — KEY LINES

```
// the only length check — at registration
if (Y.len() >= 2 &&
    Y.len() <= kMaxCachedModDivisorSize)
  SetNextCachedBigIntDivisor(y);
// promotion at call 100 — no re-check
SetCachedBigIntDivisor(y);
CachedMod_MakeInverse(Y);  // n = 50
// fixed buffer, logical length 151
GetSmallScratch_NoCheck();
scratch.set_len(scratch_space);
```

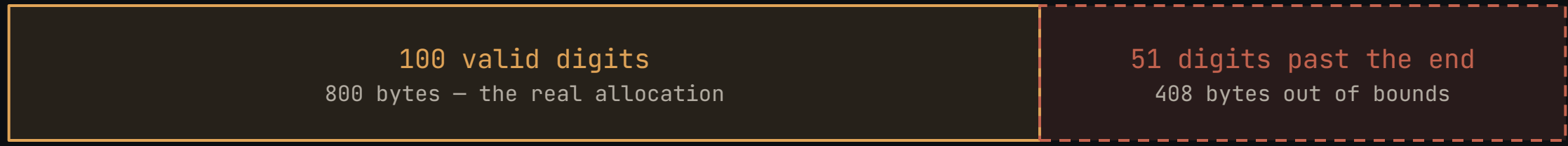
The length check guards registration. Nothing re-checks it at promotion.

CASE II — BIGINT

One buffer, two lengths

The scratch buffer as the write sink sees it — a single contiguous native allocation.

physical allocation — GetSmallScratch_NoCheck()



logical length after scratch.set_len(151) — MultiplySchoolbook() writes to all of it

The write sink is `MultiplySchoolbook()` — schoolbook multiplication over the inverse.
Written content is shaped by BigInt digits, so the attacker has real influence — but the destination is the memory right after this buffer, not an arbitrary address.

100 digits allocated, 151 digits written — a native heap overflow past one fixed buffer.

Source chain, runtime proof, exploitation

THE PIPELINE — SOURCE-LEVEL RESULT

Scout — a cross-call TOCTOU hypothesis
the divisor cache promotes on identity, without re-checking length

Sniper — traced the chain to the sink
the missing promotion-time re-check, and the fixed-size scratch buffer

Judge — confirmed a native heap OOB write
source-level confirmation, in an isolated session

RUNTIME CONFIRMATION — HUMAN POC, ASAN EXCERPT

```
AddressSanitizer: heap-buffer-overflow
WRITE of size 8
0 bytes after 800-byte region
#0 v8::bigint::MultiplySchoolbook
   bigint.h:162
   ← allocated by CachedMod_MakeInverse
```

SUBSEQUENT HUMAN EXPLOIT ENGINEERING

overflow steered onto the DateCache vptr; fake vtable in BigInt digits

```
new Date(42).toString()
RIP = 0x414141414141
```

Three stages confirm the source chain; a human PoC gives the runtime proof; the DateCache turn is later human work.

What the pipeline changed

WHAT WE GOT

GC UAF through forged read-only page identity.
The three stages identified the trusted-object check bypass behind the read-only classification; our follow-up work turned it into a trusted-space UAF, and the existing run record shows the impact.

BigInt CachedMod OOB write.
Scout proposed the cross-call cache TOCTOU, Sniper traced the missing promotion-time re-check and the fixed scratch buffer, Judge confirmed the native heap OOB write — and a human PoC reproduced it at runtime.

.....

WHAT WE TAKE FROM THIS

In these cases, the split workflow helped us find and investigate real bugs. It's wiser and more accurate.
It does not replace final verification — that still takes a human and a debugger.

SplitAI helped us generate and investigate vulnerability hypotheses.
We still relied on hands-on work to reproduce the findings and assess their impact.

Q&A